

DISK ARCHITECTURE

Module 7

DISK ARCHITECTURE

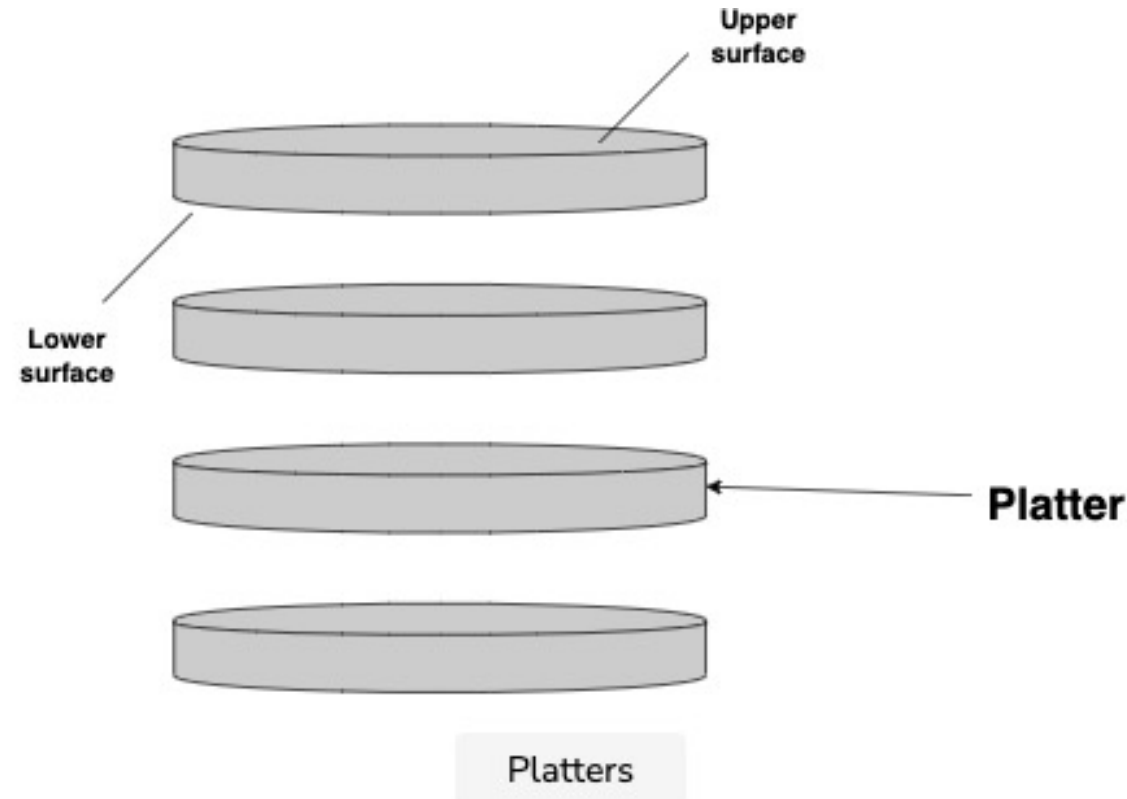
- ❖ Hard disks are secondary storage devices:
 - ❖ Use to store data.
 - ❖ The architecture of a hard disk consists of several physical components that include:
 - ❖ Platters
 - ❖ Spindle
 - ❖ Read/write heads
 - ❖ Tracks
 - ❖ Sectors
-

DISK ARCHITECTURE

- ❖ Hard disks are secondary storage devices:
 - ❖ Use to store data.
 - ❖ The architecture of a hard disk consists of several physical components that include:
 - ❖ Platters
 - ❖ Spindle
 - ❖ Read/write heads
 - ❖ Tracks
 - ❖ Sectors
-

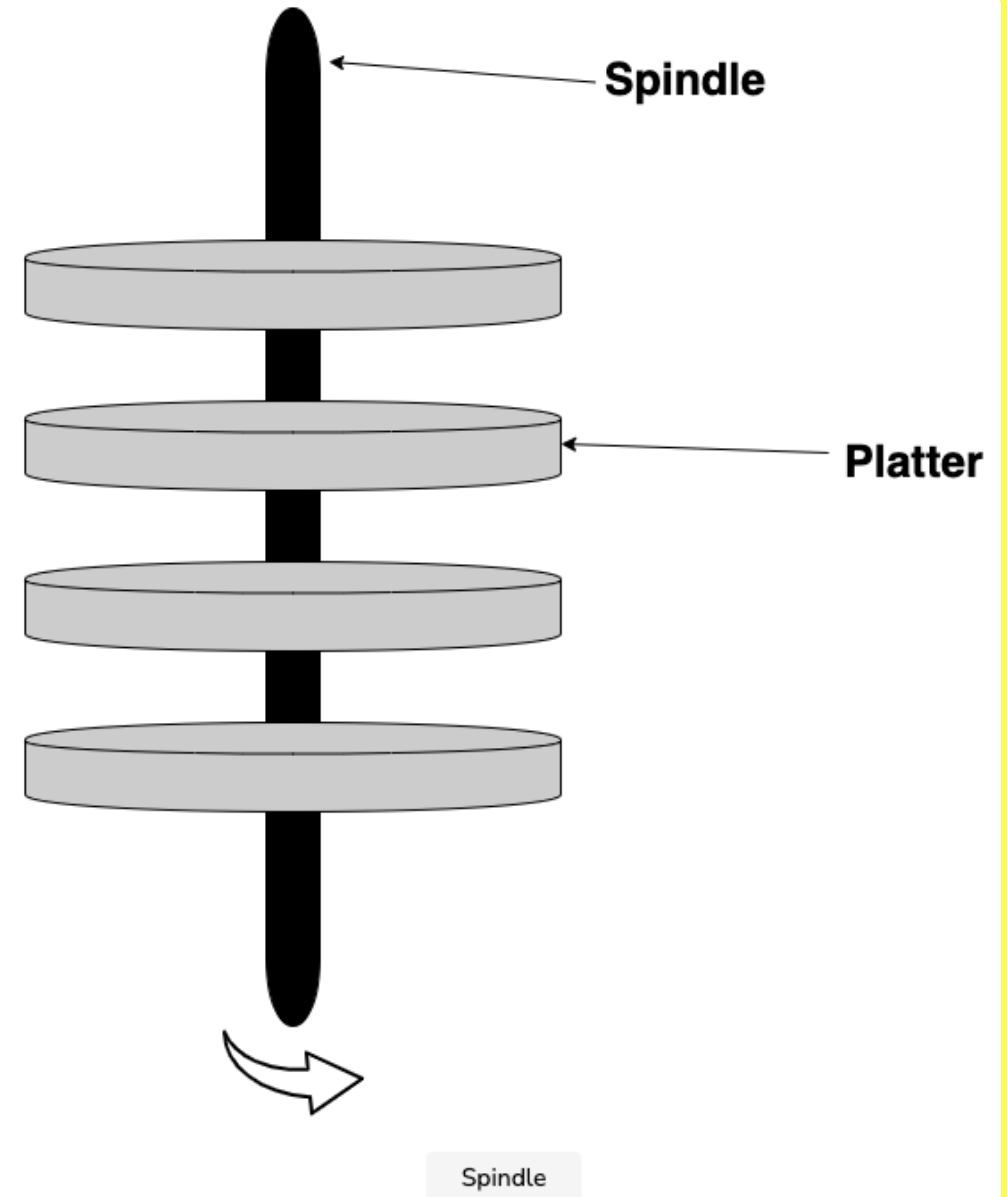
DISK ARCHITECTURE - PLATTERS

- ❖ Hard disks are organized as a concentric stack of disks.
- ❖ An individual disk is referred to as a platter.
- ❖ Each platter consists of two surfaces: a lower and an upper surface.



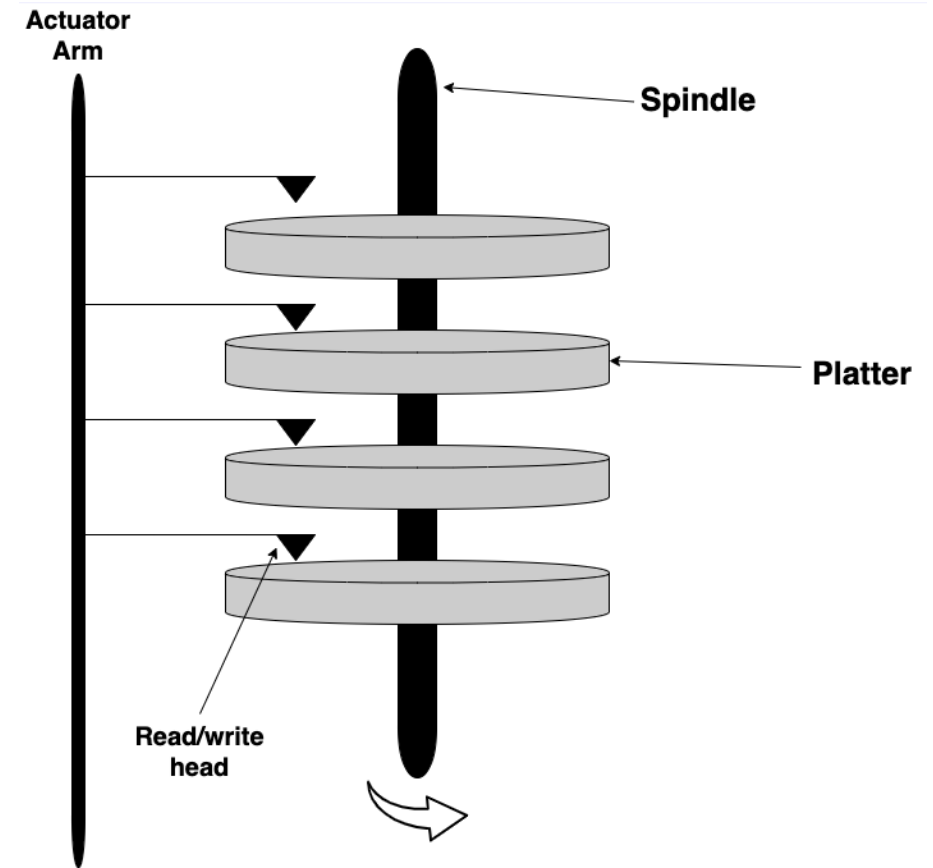
DISK ARCHITECTURE - PLATTERS

- ❖ The platters within the hard disk are connected by a spindle that runs through the middle of the platters.
- ❖ The spindle moves in a unidirectional manner along its axis (either clockwise or counterclockwise).
- ❖ The movement of the spindle causes the platters to rotate as well.



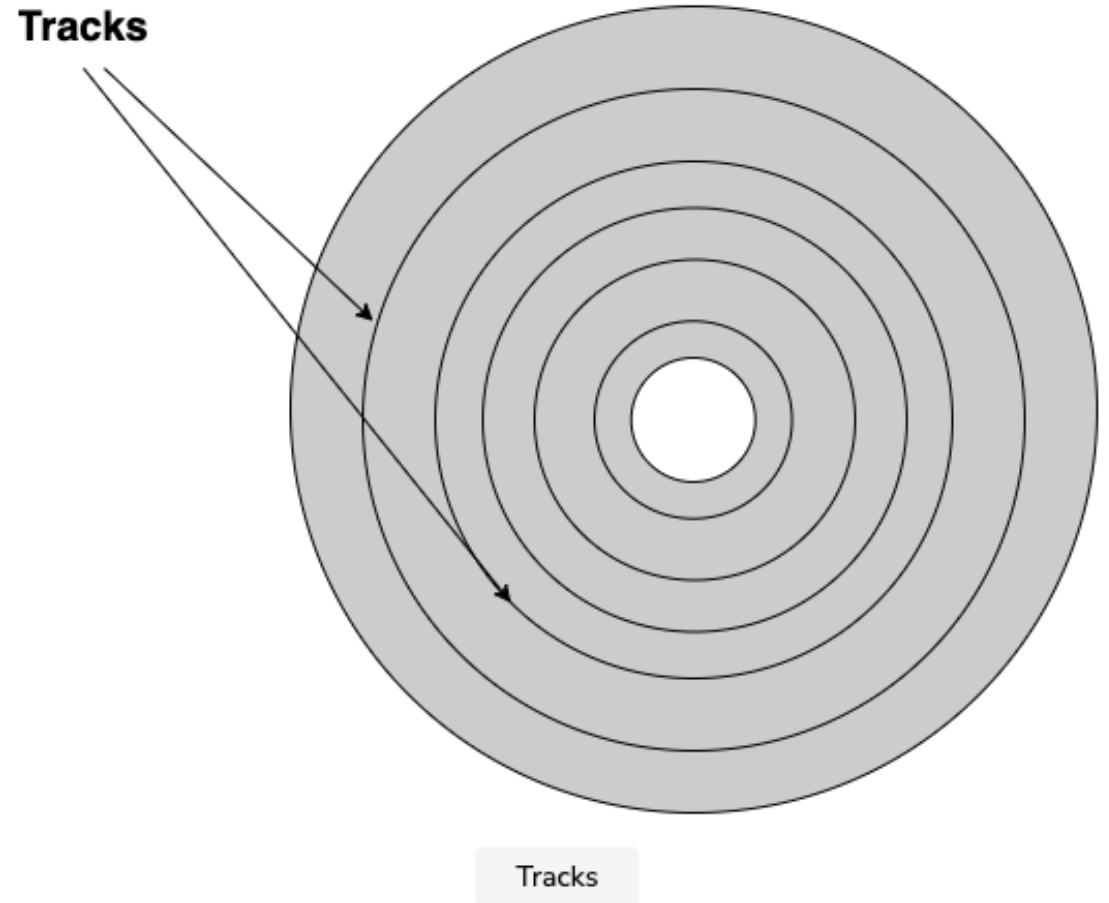
DISK ARCHITECTURE - SPINDLE

- ❖ Each surface on a platter contains a read/write head that is used to read or write data onto the disk.
- ❖ The read/write heads can move back and forth along the surface of a platter. Read/write heads are in turn connected to a single actuator arm.



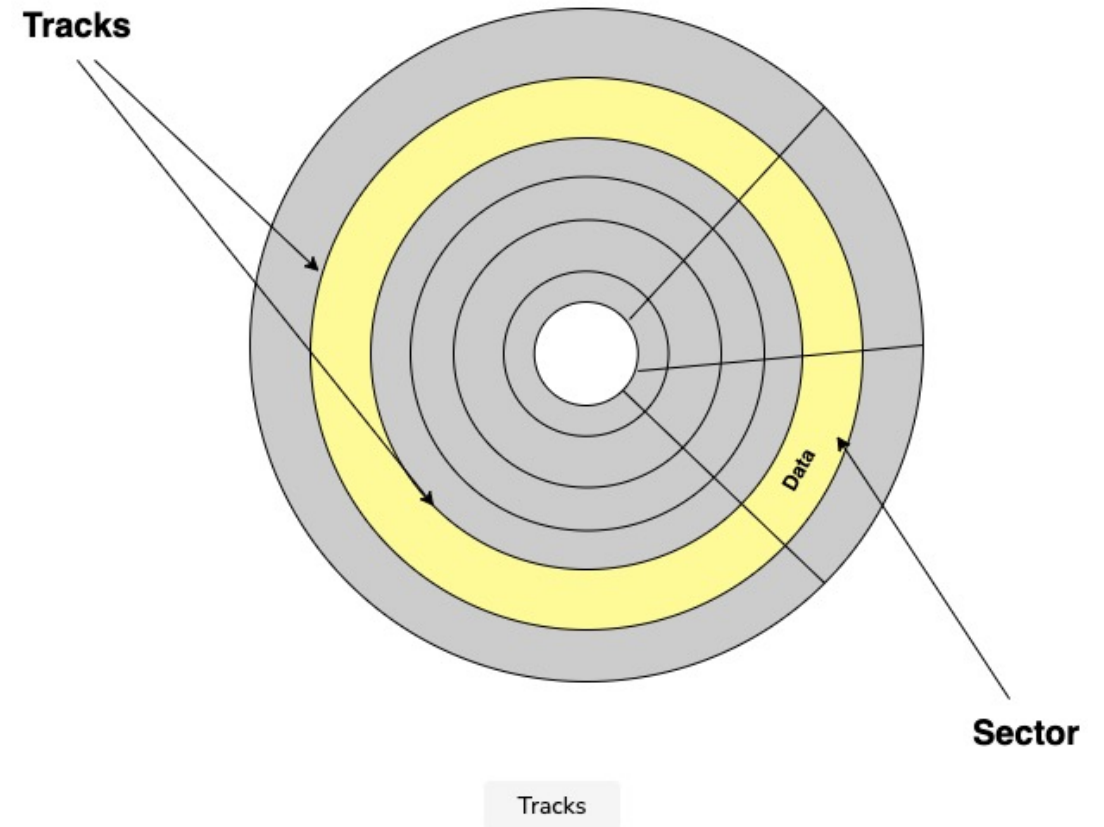
DISK ARCHITECTURE - TRACKS

- ❖ Each surface of a platter consists of a fixed number of tracks. These are circular areas on the surface of a platter that decrease in circumference as we move towards the center of the platter.
- ❖ Data is first written to the outermost track.



DISK ARCHITECTURE - SECTORS

❖ Each track is divided into a fixed number of sectors. Sectors divide track sections and store data.



DISK SCHEDULING ALGORITHMS

- ❖ An algorithm that keeps and manages input and output requests arriving for the disk in a system.
 - ❖ Analogy:
 - ❖ Consider it as a manager of a grocery store that manages all the incoming and outgoing requests for goods of that store.
 - ❖ He keeps a record of what is available in-store, what we need further, and manages the timetable of transaction of goods.
-

DISK SCHEDULING ALGORITHMS

- ❖ This algorithm helps OS to maintain an efficient manner in which input-output requests can be managed to execute a process.
 - ❖ Disk Structure in OS' is made in such a way that there are many layers of storage blocks on the disk. And when we need to access these data from disk, we initialize a 'request' to the system to give us the required data. These requests are done on a large scale.
-

DISK SCHEDULING ALGORITHMS – WHY NEEDED

- ❖ Multiple requests are coming to the disk simultaneously which will make a queue of requests.
 - ❖ This queue of requests will result in an increased waiting time of requests.
 - ❖ The requests wait until the under-processing request executes.
 - ❖ To overcome this queuing and manage the timing of these requests, 'Disk Scheduling' is important
-

DISK SCHEDULING ALGORITHMS – RELATED TERMS

- ❖ **Seek Time:** The data may be stored on various blocks of disk. To access these data according to the request, the disk arm moves and find the required block. The time taken by the arm in doing this search is known as "Seek Time".
 - ❖ **Rotational Latency:** The required data block needs to move at a particular position from where the read/write head can fetch the data. So, the time taken in this movement is known as "Rotational Latency".
 - ❖ This rotational time should be as less as possible so, the algorithm that will take less time to rotate will be considered a better algorithm.
-

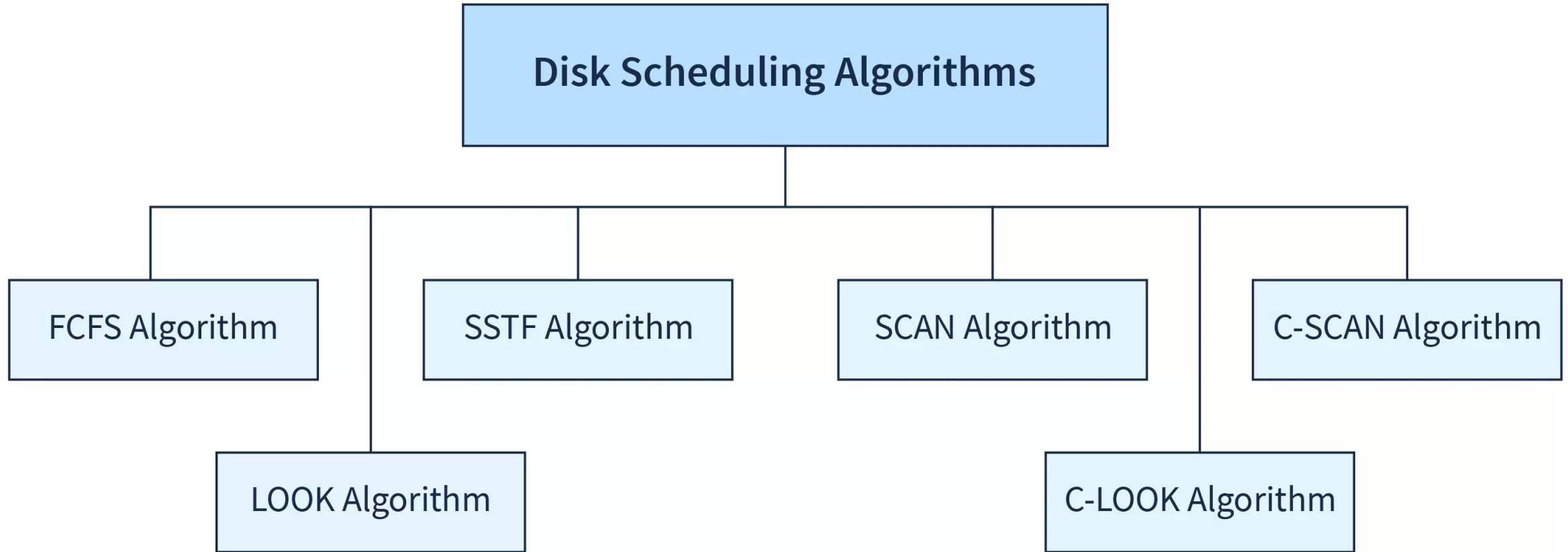
DISK SCHEDULING ALGORITHMS – RELATED TERMS

- ❖ **Transfer Time:** When a request is made from the user side, it takes some time to fetch these data and provide them as output. This taken time is known as "Transfer Time".
 - ❖ **Disk Access Time:** It is defined as the total time taken by all the above processes.
 - ❖ Disk access time = (seek time + rotational latency time + transfer time)
-

DISK SCHEDULING ALGORITHMS – RELATED TERMS

- ❖ **Disk Response Time:** The disk processes one request at a single time. So, the other requests wait in a queue to finish the ongoing process of request. The average of this waiting time is called "Disk Response Time".
 - ❖ **Starvation:** Starvation is defined as the situation in which a low-priority job keeps waiting for a long time to be executed. The system keeps sending high-priority jobs to the disk scheduler to execute first.
-

DISK SCHEDULING ALGORITHMS



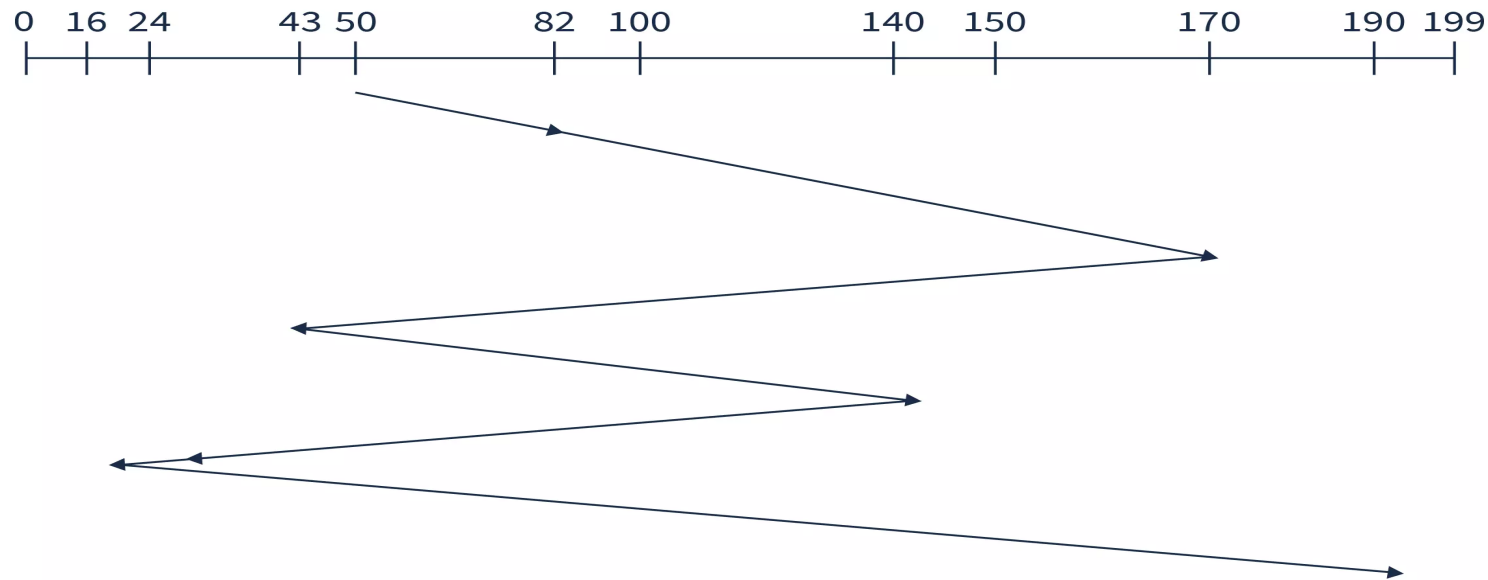
DISK SCHEDULING ALGORITHMS - FCFS DISK SCHEDULING ALGORITHM

- ❖ First-come-First-serve
 - ❖ The request which comes first will be processed first and so on.
 - ❖ The requests coming to the disk are arranged in a proper sequence as they arrive.
 - ❖ no chance of 'starvation'
-

DISK SCHEDULING ALGORITHMS - FCFS DISK SCHEDULING ALGORITHM

❖ Example:

❖ Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



DISK SCHEDULING ALGORITHMS - FCFS DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ The head starts at position 50 and moves to request 82.
- ❖ Then moves towards the second request that is 170 and then to the request 43 and so on.
- ❖ The disk arm will serve the requests in arriving order.
- ❖ **Seek time**= $(82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) = 642$

DISK SCHEDULING ALGORITHMS - FCFS DISK SCHEDULING ALGORITHM

❖ Advantages:

- ❖ Implementation is easy.
- ❖ No chance of starvation.

❖ Disadvantages:

- ❖ 'Seek time' increases.
 - ❖ Not so efficient.
-

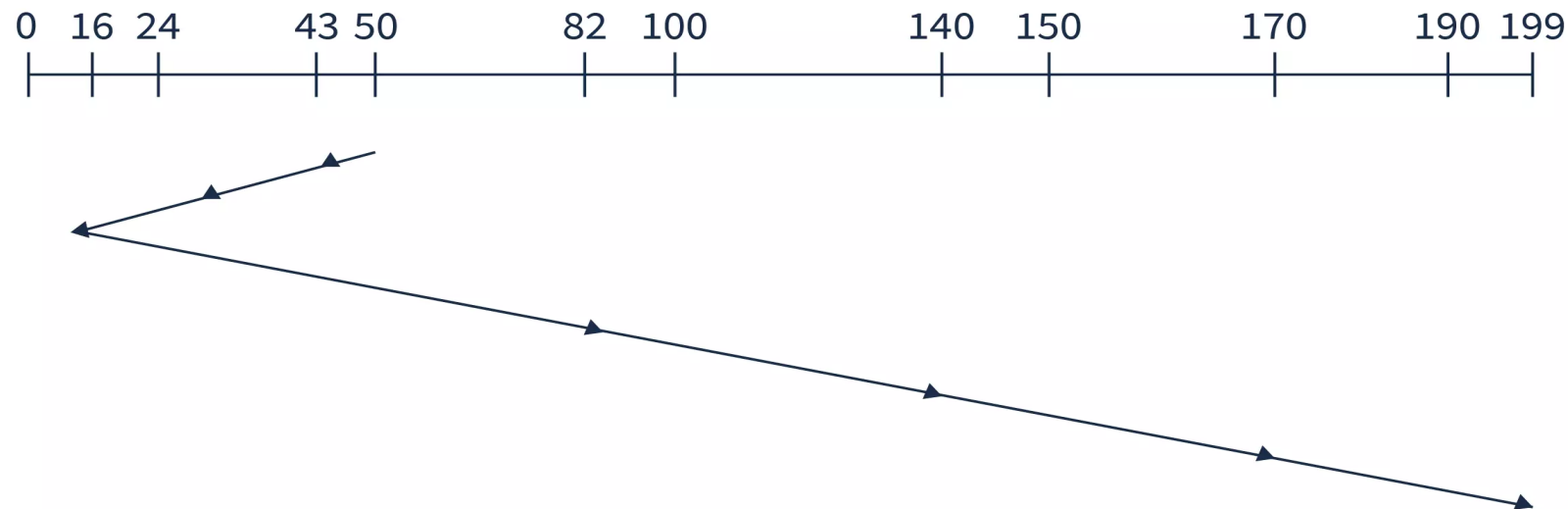
DISK SCHEDULING ALGORITHMS - SSTF DISK SCHEDULING ALGORITHM

- ❖ Shortest seek time first
 - ❖ It searches for the request having the least 'seek time' and executes them first.
 - ❖ This algorithm has less 'seek time' as compared to FCFS Algorithm.
-

DISK SCHEDULING ALGORITHMS - SSTF DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



DISK SCHEDULING ALGORITHMS - SSTF DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ The disk arm searches for the request which will have the least difference in head movement.
 - ❖ The least difference is (50-43).
 - ❖ After 43, the head will be nearest to 24, and from here the head will be nearest to the request 16, After 16, the nearest request is 82, so the disk arm will move to serve request 82 and so on.
 - ❖ $\text{Seek Time} = (50-43) + (43-24) + (24-16) + (82-16) + (140-82) + (170-140) + (190-170) = 208$
-

Advantages:

- ❖ Disk response time is less.
- ❖ More efficient than FCFS.

❖Disadvantages:

- ❖ Less speed of algorithm execution.
 - ❖ Starvation can be seen.
-

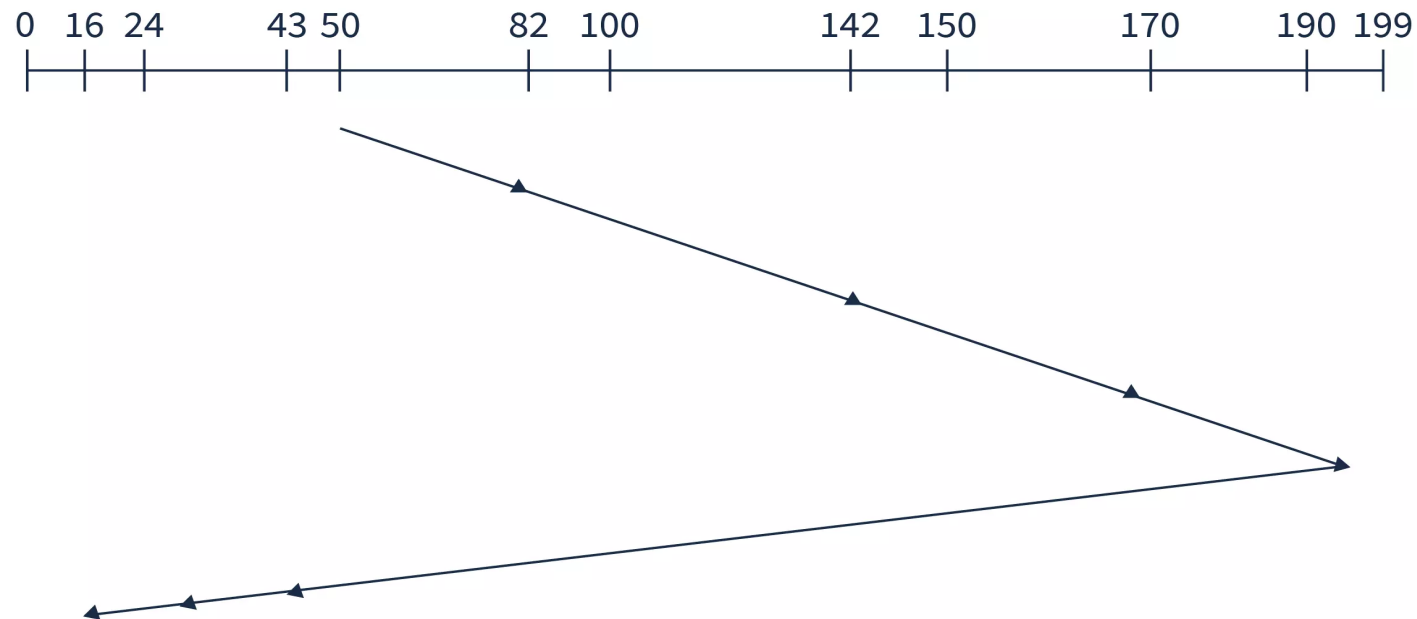
DISK SCHEDULING ALGORITHMS - SCAN DISK SCHEDULING ALGORITHM

- ❖ The head starts to scan all the requests in a direction and reaches the end of the disk.
 - ❖ It reverses its direction and starts to scan again the requests in its path and serves them.
 - ❖ It is also known as the "Elevator Algorithm".
-

DISK SCHEDULING ALGORITHMS - SCAN DISK SCHEDULING ALGORITHM

❖ Example:

❖ Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50. The 'disk arm' will first move to the larger values.



DISK SCHEDULING ALGORITHMS - SCAN DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ The disk arm starts from position 50 and goes in a single direction until it reaches the end of the disk i.e.- request position 199.
 - ❖ After that, it reverses and starts servicing in the opposite direction until reached the other end of the disk.
 - ❖ This process keeps going on until the process is executed.
 - ❖ Seek Time' will be like: $(199-50) + (199-16) = 332$
-

DISK SCHEDULING ALGORITHMS - SCAN DISK SCHEDULING ALGORITHM

❖ Advantages:

- ❖ Implementation is easy.
- ❖ Requests do not have to wait in a queue.

❖ Disadvantage:

- ❖ The head keeps going on to the end even if there are no requests in that direction.
-

DISK SCHEDULING ALGORITHMS – C-SCAN DISK SCHEDULING ALGORITHM

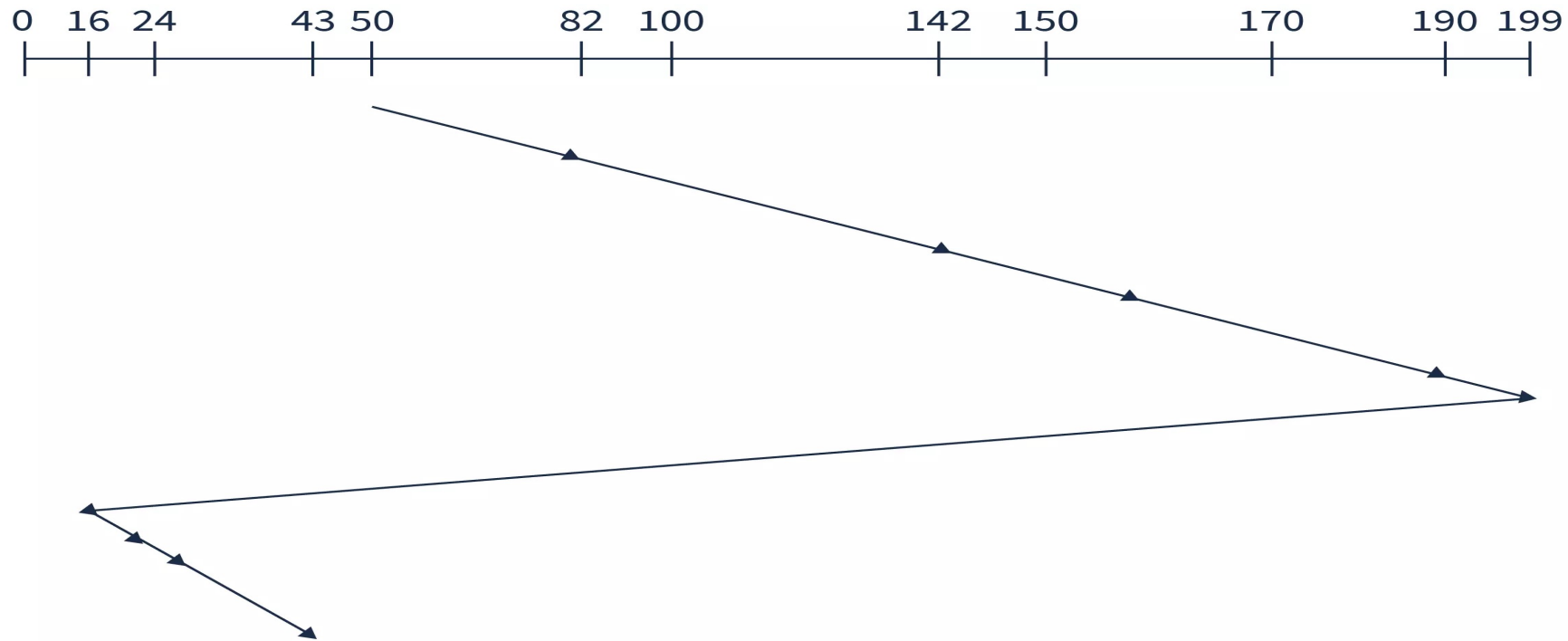
❖ Circular-Scan

- ❖ Almost the same as the Scan disk algorithm but one thing that makes it different is that 'after reaching the one end and reversing the head direction, it starts to come back.
 - ❖ The disk arm moves toward the end of the disk and serves the requests coming into its path
 - ❖ After reaching the end of the disk it reverses its direction and again starts to move to the other end of the disk but **while going back it does not serve any requests.**
-

DISK SCHEDULING ALGORITHMS – C-SCAN DISK SCHEDULING ALGORITHM

❖ Example:

❖ Suppose a disk having 200 tracks (0-199). The request sequence (82, 170, 43, 140, 24, 16, 190) are shown in the given figure and the head position is at 50.



DISK SCHEDULING ALGORITHMS – C-SCAN DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ The disk arm starts from position 50 and reached the end(199) and serves all the requests in the path.
 - ❖ Then it reverses the direction and moves to the other end of the disk i.e.- 0 without serving any task in the path.
 - ❖ After reaching 0, it will again go move towards the largest remaining value which is 43.
 - ❖ the head will start from 0 and moves to request 43 serving all the requests coming in the path.
 - ❖ Seek Time will be $= (199-50) + (199-0) + (43-0) = 391$
-

DISK SCHEDULING ALGORITHMS – C-SCAN DISK SCHEDULING ALGORITHM

❖ Advantages:

- ❖ The waiting time is uniformly distributed among the requests.
- ❖ Response time is good in it.

❖ Disadvantages:

- ❖ The time taken by the disk arm to locate a spot is increased here.
 - ❖ The head keeps going to the end of the disk.
-

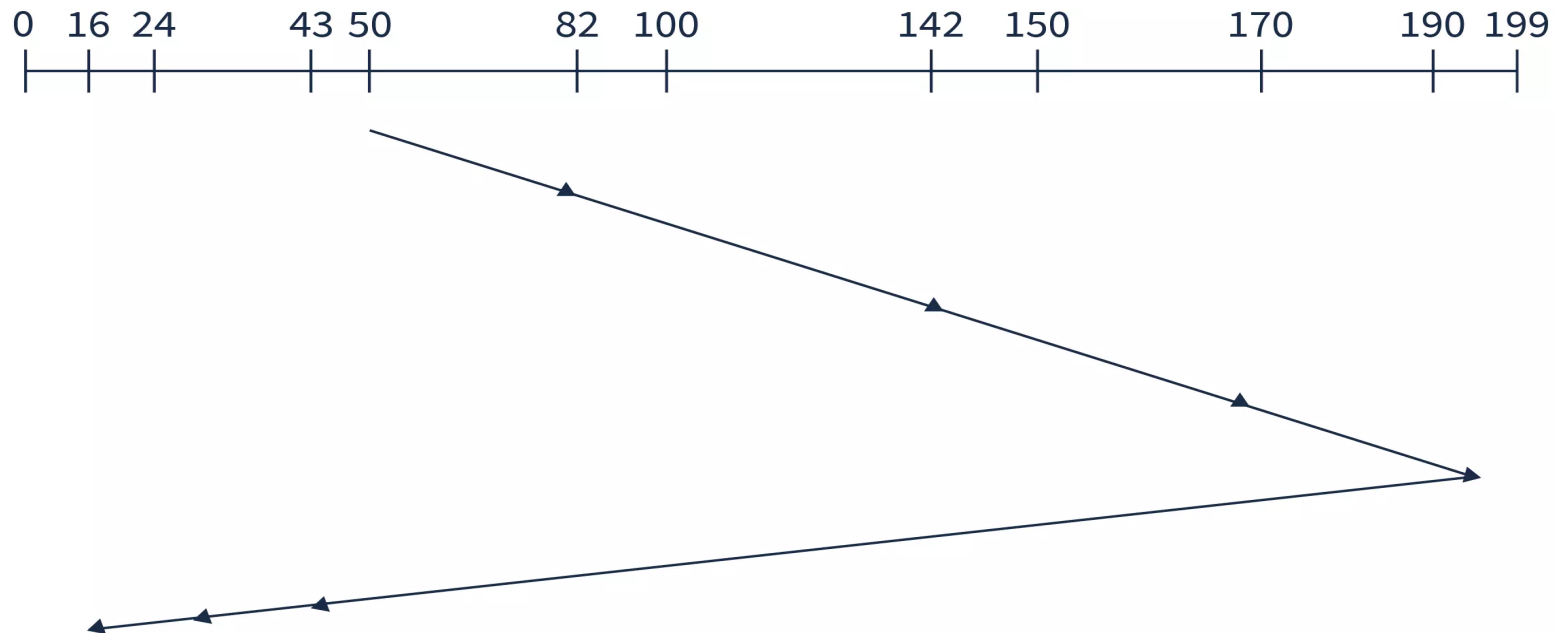
DISK SCHEDULING ALGORITHMS – LOOK DISK SCHEDULING ALGORITHM

- ❖ The disk arm moves to the 'last request' present and services them.
 - ❖ After reaching the last requests, it reverses its direction and again comes back to the starting point.
 - ❖ It does not go to the end of the disk, in spite, it goes to the end of requests
-

DISK SCHEDULING ALGORITHMS – LOOK DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ Disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



DISK SCHEDULING ALGORITHMS – LOOK DISK SCHEDULING ALGORITHM

❖ Example:

- ❖ The disk arm is starting from 50 and starts to serve requests in one direction only but in spite of going to the end of the disk, it goes to the end of requests i.e.-190.
 - ❖ Then comes back to the last request of other ends of the disk and serves them
 - ❖ And again, starts from here and serves till the last request of the first side.
 - ❖ Seek time $= (190 - 50) + (190 - 16) = 314$
-

DISK SCHEDULING ALGORITHMS – LOOK DISK SCHEDULING ALGORITHM

❖ Advantages:

- ❖ Starvation does not occur.
- ❖ Since the head does not go to the end of the disk, the time is not wasted here.

❖ Disadvantage:

- ❖ The arm has to be conscious to find the last request
-

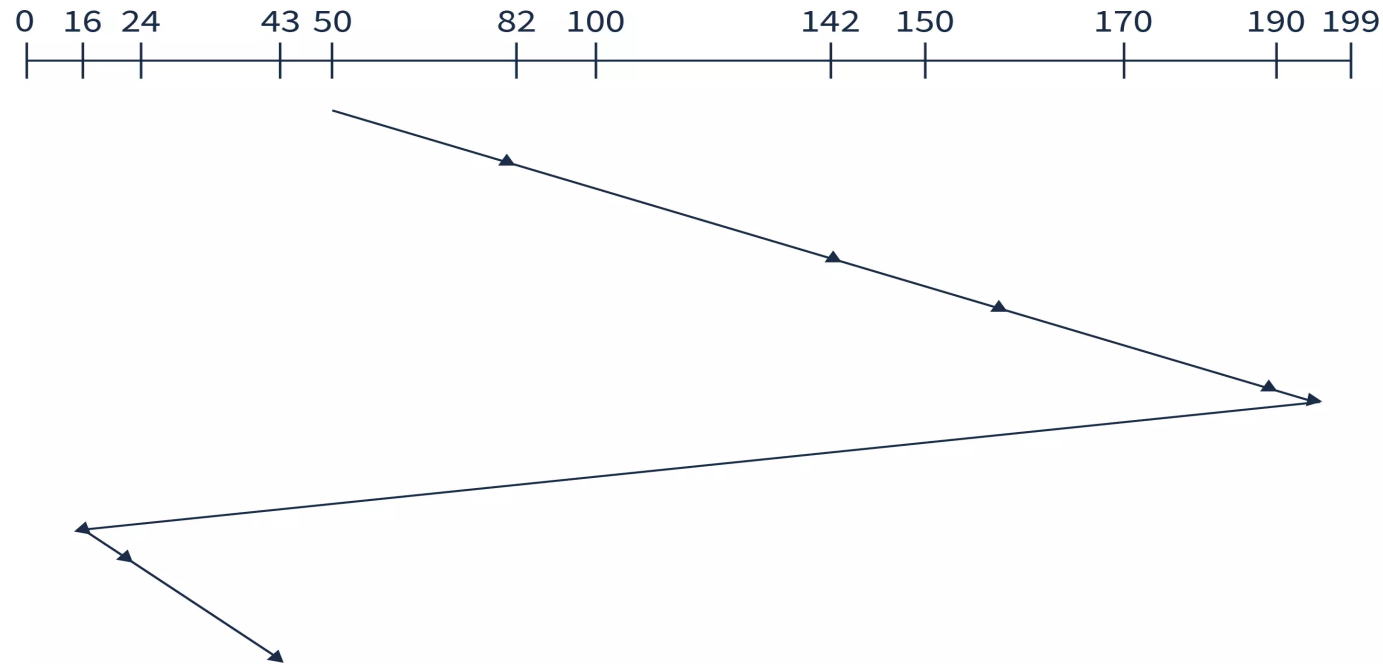
DISK SCHEDULING ALGORITHMS – CLOOK DISK SCHEDULING ALGORITHM

- ❖ Almost the same as the Look algorithm
 - ❖ After reaching the end requests, it reverses the direction of the head and starts moving to the initial position.
 - ❖ But in moving back, it does not serve any requests.
-

DISK SCHEDULING ALGORITHMS – CLOOK DISK SCHEDULING ALGORITHM

❖ Example

- ❖ Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



DISK SCHEDULING ALGORITHMS – CLOOK DISK SCHEDULING ALGORITHM

❖ Example

- ❖ The disk arm is starting from 50 and starts to serve requests in one direction only but in spite of going to the end of the disk, it goes to the end of requests i.e.-190.
 - ❖ Then comes back to the last request of other ends of a disk without serving them
 - ❖ Again starts from the other end of the disk and serves requests of its path.
 - ❖ $\text{Seek Time} = (190-50) + (190-16) + (43-16) = 341$
-

DISK SCHEDULING ALGORITHMS – CLOOK DISK SCHEDULING ALGORITHM

❖ Advantages:

- ❖ The waiting time is decreased.
- ❖ If there are no requests till the end, it reverses the head direction immediately.
- ❖ Starvation does not occur.
- ❖ time taken by the disk arm to find the desired spot is less.

❖ Disadvantage:

- ❖ The arm has to be conscious about finding the last request.
-